

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

C02: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance (BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

Annexure A

Debugging or Optimization task

Code of Conduct:

I ANGEL DEEPIKA M (192521272) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Angel Deepika M

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2) critical
section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1) critical
section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock.

Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready. You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB
- Heap: 150 MB

- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead. Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Q6. Producer–Consumer Debugging (Semaphore Error)

A producer-consumer application occasionally hangs after running for several minutes.

Semaphore empty = 5
Semaphore full = 0
Semaphore mutex = 1

Producer: wait(mutex)
wait(empty)
insert(item)
signal(full)
signal(mutex)

Consumer:
wait(mutex) wait(full)
remove(item)
signal(empty)
signal(mutex)

The program freezes even though both producer and consumer threads are alive.

1. Identify the synchronization error causing the hang.
2. Explain why the semaphore ordering is incorrect.
3. Suggest the correct semaphore sequence to eliminate the problem.

(10 Marks)

Q7. Thread Performance Debugging

A web server creates one new thread for every incoming client request.

Performance statistics:

- Average requests/sec = 15,000
- Average thread creation time = 0.4 ms • CPU utilisation = 92%
- Memory usage continuously increases

Developers notice that the server becomes slow after several hours.

1. Identify the thread-management problem.
2. Explain why the current multithreading model causes performance degradation.
3. Propose two optimisations to improve scalability.

(10 Marks)

Q8. Banker's Algorithm Debugging

Available Resources:

A B C = 2 1 1

Allocation:

P1: 1 0 1

P2: 0 1 0

P3: 1 1 1

Maximum:

P1: 2 1 2

P2: 1 2 1

P3: 3 2 2

Process P2 requests (1,1,1).

1. Calculate the Need matrix.
2. Determine whether the request can be granted immediately using Banker's Algorithm.
3. If not, identify the unsafe condition and explain why granting the request could lead to deadlock.

(10 Marks)

Q9. Deadlock Detection and Recovery Debugging

A database server reports that four transactions are waiting for resources.

Resource Allocation Graph:

- T1 waits for R2
- T2 waits for R3
- T3 waits for R4
- T4 waits for R1
- R1 allocated to T1
- R2 allocated to T2
- R3 allocated to T3
- R4 allocated to T4

The database becomes completely unresponsive.

1. Identify whether a deadlock exists.
2. Explain how deadlock detection identifies this situation.

3. Suggest two recovery techniques and justify which one minimises system disruption.

(10 Marks)

Q10. Monitor-Based Synchronization Debugging

A monitor is used to synchronize access to a shared printer.

```
monitor Printer
{
  boolean busy =
  false;
```

```
  procedure Print()
  {
    if(busy)
    wait();
```

```
    busy = true;
```

```
    // Printing
```

```
    busy = false;
  }
}
```

Users report that some print jobs never execute even though the printer becomes free.

1. Identify the logical error in the monitor implementation.
2. Explain why waiting processes are never awakened.
3. Modify the monitor logic to ensure correct synchronization and fairness.

(10 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging(25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (100)
Q1						
Q2						
Q3						
Q4						

Q5						
Q6						
Q7						
Q8						
Q9						
Q10						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1.

Deadlock Debugging :-

Problem :-

Process P_1 locks R_1 first and waits for R_2 .

Process P_2 locks R_2 first and waits for R_1 .

Each process is waiting for a resource held by the other.

Resource R_1 = Free

Resource R_2 = Free.

Step	Process P_1	Process P_2
1.	lock(R_1)	
2.		lock(R_2)
3.	Wait (100ms)	Wait (100ms)
4.	Tries to lock (R_2) $\rightarrow$ Blocked	Tries to lock (R_1) $\rightarrow$ Blocked

This creates a circular wait which leads to deadlock.

Optimized code:

Process P1:

lock (R1)

lock (R2)

critical section

unlock (R2)

unlock (R1)

Process P2:

lock (R1)

lock (R2)

critical section

unlock (R2)

unlock (R1)

Result:

Both processes request resources in the same order,
so deadlock is eliminated.

2. Cpu utilisation problem:-

Observation:

Cpu utilisation = 30 %

Schedule log:

Ready $\rightarrow$ Waiting $\rightarrow$ Ready $\rightarrow$ Waiting $\rightarrow$ Ready.

Debugging:-

Cpu executes process



Needs I/O.



Moves to Waiting.



Cpu becomes idle



I/O completes



Returns to Ready queue.

Root cause:

Frequent I/O waiting causes the cpu to remain idle.

Optimization:

Increase multiprogramming
use asynchronous I/O.

Improve Cpu Scheduling.

Result:

Cpu utilisation increases because another process executes while one waits for I/O.

3. Memory Bottleneck Debugging :-

Given:-

Code = 5 MB.

Heap = 150 MB.

Stack = 1 MB per thread

Threads = 25.

RAM = 2 GB = 2048 MB.

Calculate memory:

Stack memory.

$$= 25 \times 1$$

$$= 25 \text{ MB.}$$

Total memory used

$$= \text{code} + 150 + 25$$

$$= 180 \text{ MB.}$$

Debugging:

Although only 180 MB is used, many active threads increase page faults and memory access overhead.

Optimization:

1. Uses a thread pool
2. Improve memory locality & reduce unnecessary allocations

Result:

Page faults decrease and execution becomes faster.

4.

Starvation Debugging :-

Queue Structure :-

- $Q_1 \rightarrow$ Interactive (highest priority)
- $Q_2 \rightarrow$ System
- $Q_3 \rightarrow$ Batch (P5)

Execution :-

Interactive jobs continuously arrive
CPU always executes Q_1
P5 remains waiting in Q_3 .

Root cause :-

Strict priority scheduling causes starvation.

Optimization :-

Apply Aging

Eg: Waiting time $\uparrow$
Priority $\uparrow$

P5 moves higher.

Result :-

P5 gets CPU time and starvation is removed.

Context Switching Overhead :-

Given :-

Context Switches = 20,000/sec

Time Quantum = 1ms

Burst time = 2ms.

Debugging :-

Thread starts



Runs for 1ms



Preempted



Context switch



Runs again



Another context switch.

Root cause :-

Small time Quantum causes excessive context switching

Result :

Context switches decrease and cpu efficiency improves.

6. Procedure Consumer Debugging :-

Incorrect Sequence :-

Producer :-

Wait (mutex)

Wait (empty)

Consumer :

Wait (mutex)

Wait (full)

Debugging :-

Suppose buffer is full.

Producer acquires mutex.

Producer waits on empty.

Consumer needs mutex to remove an item.

Producer is waiting

Consumer is blocked

System freezes

Correct Sequence :-

Producer :

Wait (empty)

Wait (mutex)

Insert (item)

Signal (mutex)

Signal (full) .

consumer :

wait (full)

wait (mutex)

remove (item)

signal (mutex)

signal (empty) .

Result :-

No deadlock occurs because mutex is acquired only after resource availability is confirmed .

Thread performance Debugging :-

Given :-

Requests / sec = 15,000

Thread creation = 0.4 ms .

Cpu = 92% .

Memory continuously increases .

Debugging :-

Request



Create thread .

Execute



Destroy thread.

This repeats thousands of times.

Root cause:

Frequent thread creation causes:

- High CPU overhead.
- High memory usage
- Increased Context Switching.

Optimization

- Thread pool
- Reuse worker threads.

Result:

Lower CPU usage and improved scalability.

8. Banker's Algorithm:-

Need matrix:

Need = maximum - Allocation.

Process	maximum	Allocation	Need .
P ₁	(2, 1, 2)	(1, 0, 1)	(1, 1, 1)
P ₂	(1, 2, 1)	(0, 1, 0)	(1, 1, 1)
P ₃	(3, 2, 2)	(1, 1, 1)	(2, 1, 1)

Check Request :

Available = (2, 1, 1)

Request = (1, 1, 1).

$(1, 1, 1) \leq \text{Need} \checkmark$

$(1, 1, 1) \leq \text{Available} \checkmark$

Temporary Available.

(2, 1, 1)

-(1, 1, 1)

= (1, 0, 0)

Safety check :-

P₁ needs (1, 1, 1) X

P₂ needs (1, 1, 1) X

P₃ needs (2, 1, 1) X

No process can finish.

2. Deadlock Detection and Recovery :-

Resource Allocation Graph :-

$T_1 \rightarrow R_2 \rightarrow T_2$

$\downarrow$

$R_1 \leftarrow T_4 \leftarrow R_4 \leftarrow T_3 \leftarrow R_3$

Debugging :-

cycle found :-

T_1

$\downarrow$

T_2

$\downarrow$

T_3

$\downarrow$

T_4

$\downarrow$

T_1

cycle exist

Deadlock exist .

Methods:-

Terminate one transaction.

Rollback one transaction.

Best method:-

Rollback causes less data loss and less system disruption.

Result:-

Deadlock is removed by breaking the cycle.

10. Monitor-Based Synchronization:-

Incorrect logic:-

busy = False.

No, waiting process is notified.

Debugging:-

Thread A prints

Thread B waits.

Printer becomes free.

No signal is sent.

Thread B waits forever.

Correct monitor :-

Monitor printer.

```
{  
    boolean busy = false;  
    Procedure print ()  
    {  
        While (busy)  
            wait();  
        busy = true;  
        busy = false;  
        Signal ();  
    }  
}
```

Result :-

Waiting process are awakened correctly, ensuring synchronization and fairness